

Research and Planning

Specifications

- Target Market: General Purpose Computer
- Memory Organization: Von Neumann architecture
- Address Bus Width: 14 bits
- Number of Addresses 16384
- Program Load Address: 0x2000
- Data Bus Width: 36 bits
- Number of Register File Address Bits: 2
- Number Register File Registers: 4
- Width of Register File Register : 36
- Instruction Width: 18 bits
- Size of Immediate Field: 8 bits
- Maximum number of Instructions: 16
- Memory Addressing Endianness : Little

Phase 1 - Instruction Set Design

You will need to design and specify your instruction set, that is, you will need to determine your command set. How will you define the fields for each instruction? You need to determine whether you want to use two or three operand codes. How will your registers be named and defined? You are free to borrow ideas from MIPS, x86, RiscV, PowerPC, and any computer Assembly Language. You will need to provide Verilog on how to decode your instructions. For an example, here is a slice of the MIPS architecture:

```
module instruction_fields(  
    input [31:0] instruction,  
    output [4:0] Register_Rs,  
    output [4:0] Register_Rt,  
    output [4:0] Register_Rd,  
    output [15:0] immediate ) ;  
    assign Register_Rs = instruction[25:21];  
    assign Register_Rt = instruction[20:16];  
    assign Register_Rd = instruction[15:11];
```

```
assign immediate = instruction[15:0];
```

```
endmodule
```

Phase 2 - Implementation in Verilog and Simulation

In the second phase of the project, you will implement the various Verilog modules which will implement your design. You may use Xilinx ISE, iVerilog, EDA Playground, Synopsys, or Cadence tools to simulate your design. Your implementation will be tested on the two programs above.

Notes

This document will be updated with more details and information as needed.

Von Neumann Architecture

Pipelining

3 stage pipeline

<https://www.allaboutvlsi.com/interview-question-fetch-decode-execute-processor>

Pipelining is analogous to an assembly line.

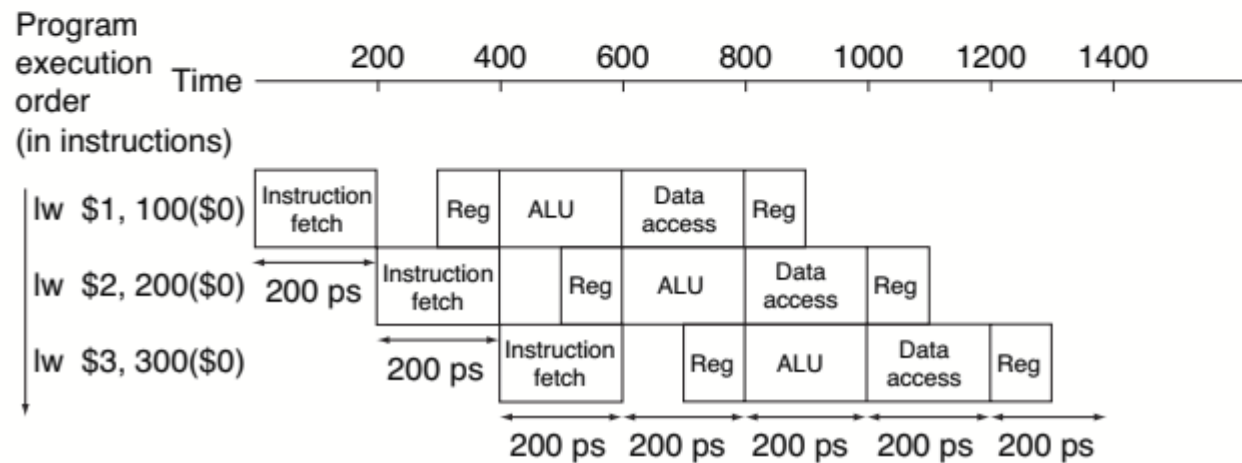
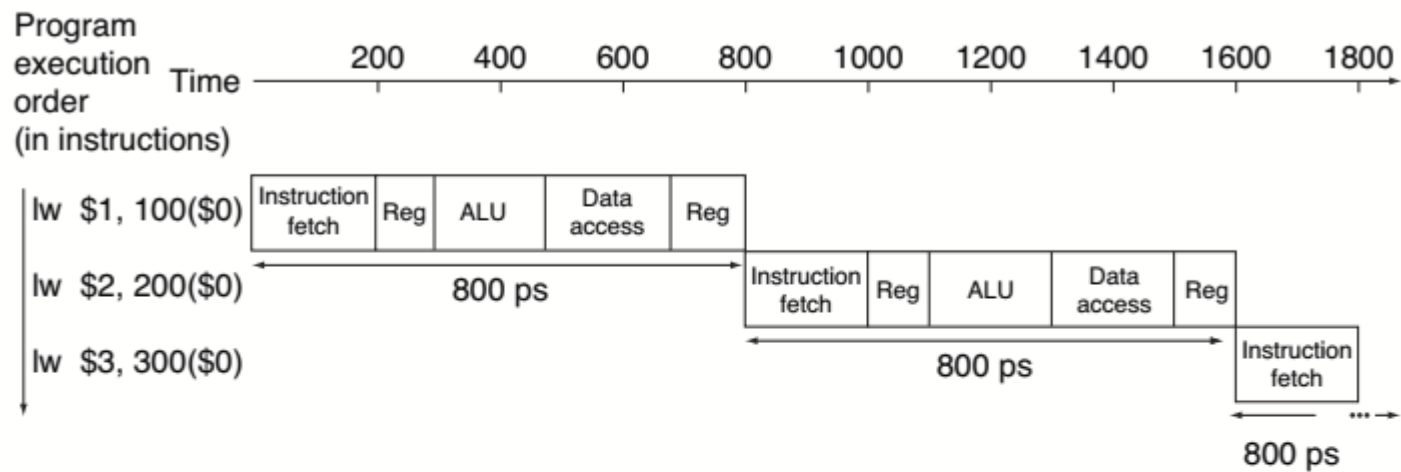
Pipelining decreases the total time for multiple loads/instructions but does not decrease the total time for one load.

MIPS breaks the pipeline into 5 steps:

- Fetch instruction from memory (Fetch)
- Decode the instruction (Decode)
- Execute the operation or calculate address (Execute)
- Access an operand in data memory (Read)
- Write result into register (Write)

3 stage pipelining on the other hand combines the read/write operation with the execute stage. It reduces number of hazards to deal with and simplifies control logic but it decreases throughput since the instruction time for the execute stage increases.

$$\text{Time between instructions}_{\text{pipelined}} = \frac{\text{Time between instructions}_{\text{nonpipelined}}}{\text{Number of pipe stages}}$$

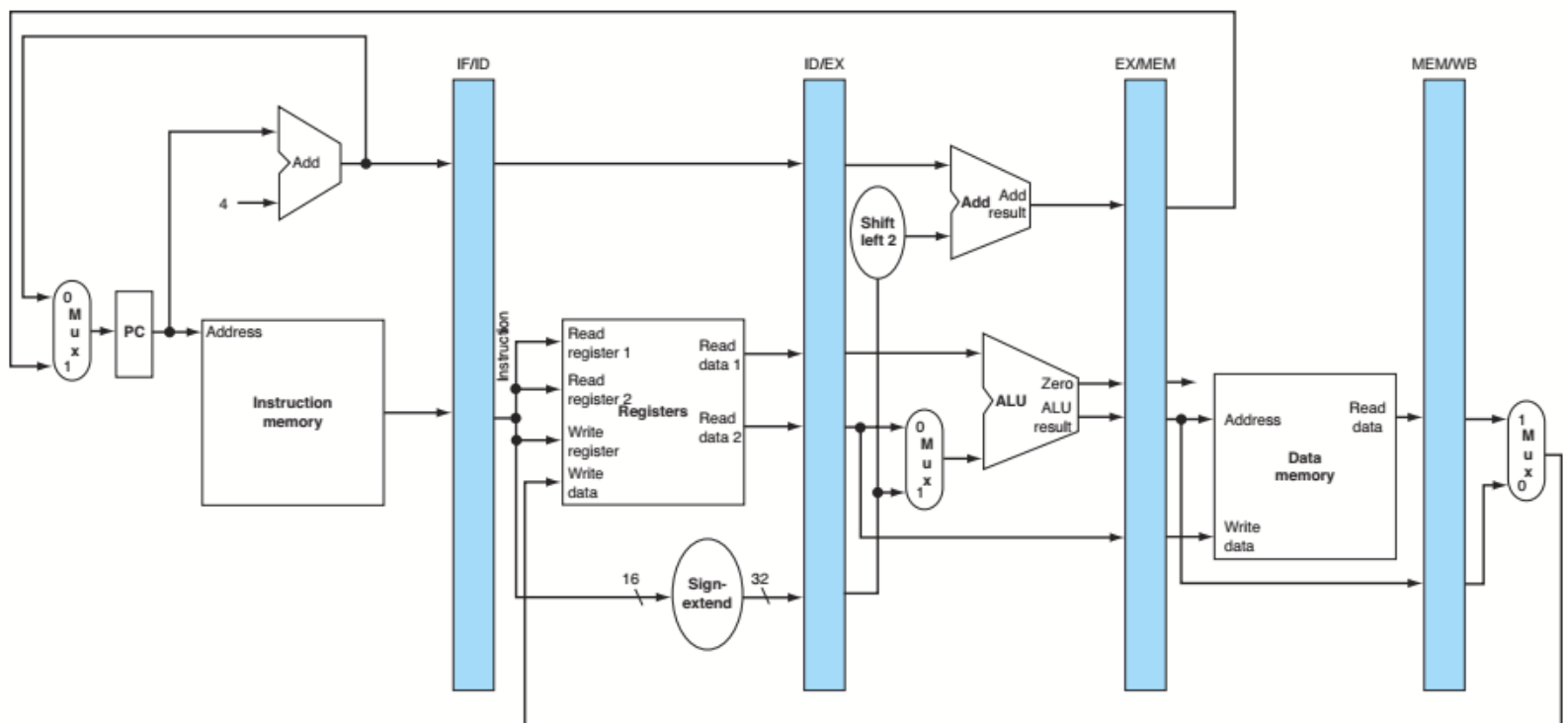
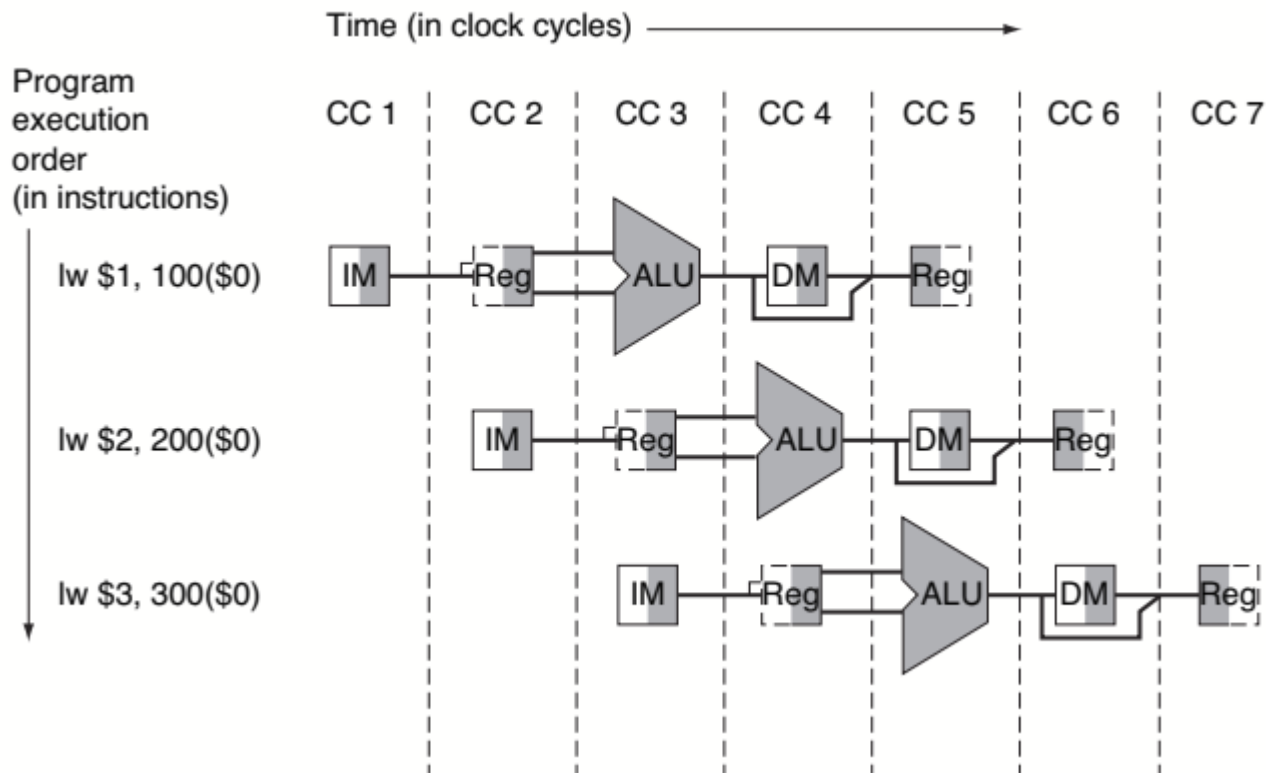


Design Instruction Sets for Pipelining

- All instructions in MIPS are same-length.
- MIPS only has few instruction formats
 - symmetry means second stage can start reading register file at same time instruction was fetched.
- Memory operands only appear in load or stores in MIPS
 - Allows us to calculate memory addresses and then access memory in following stage
- Operands align in memory
 - do not need to worry about single data instruction requiring two memory accesses

Outside the memory system, the effective operation of the pipeline is usually the most important factor in determining the CPI of the processor and hence its performance.

We must separate the data-path into 5 pieces, each piece representation a stage of the pipeline



To retain value of instruction for other 4 stages we must save it in a pipeline register indicated by each blue dividing rectangle.

The registers must be wide enough to store all the data corresponding to the lines that go through them.

For example, the IF/ID register must be 64 bits wide, because it must hold both the 32-bit instruction fetched from memory and the incremented 32-bit PC address. We will expand these registers over the course of this chapter, but for now the other three pipeline registers contain 128, 97, and 64 bits, respectively.

The program counter can be thought of as a pipeline register to the IF stage. Essential part of architecture, must save address when exception occurs.

we need to preserve the destination register number in the load instruction because a load needs to know the register number of the destination.

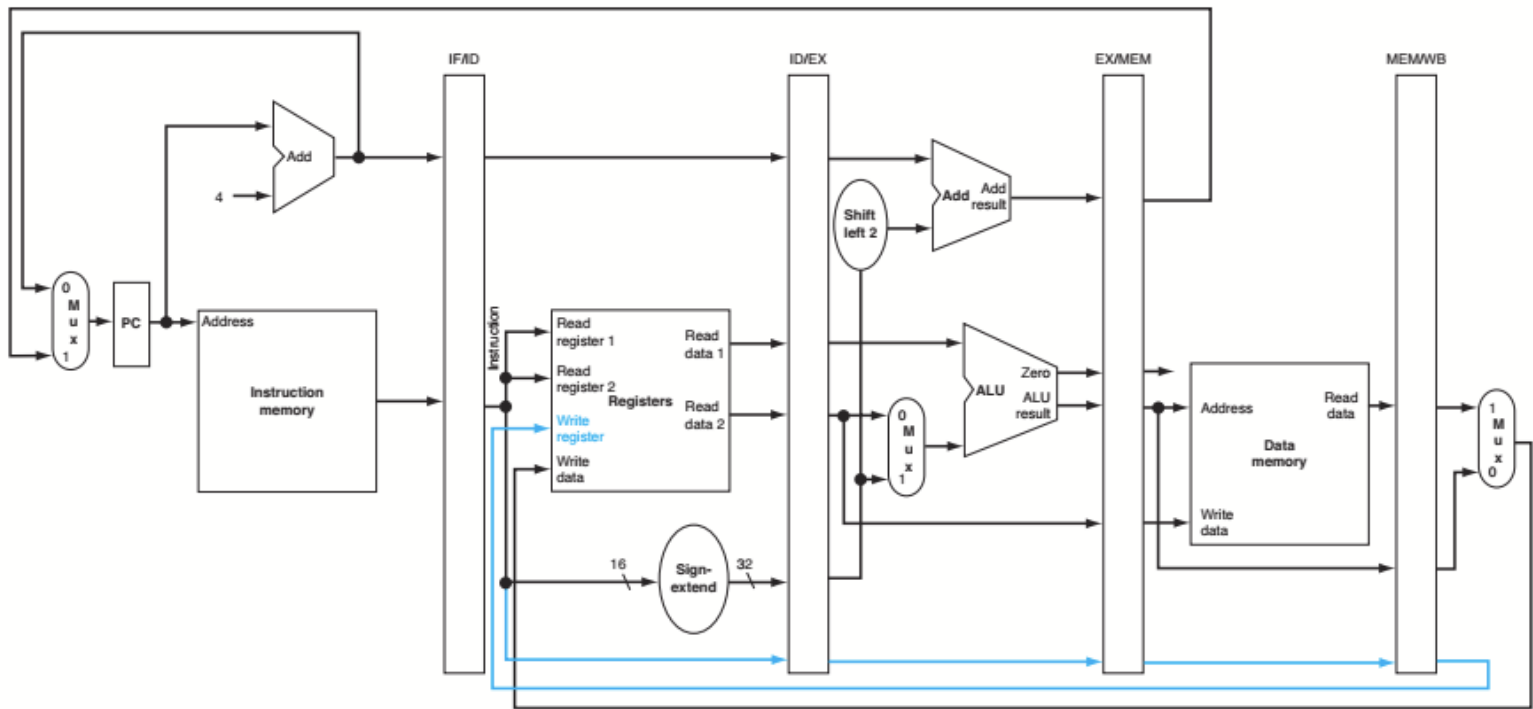


FIGURE 4.41 The corrected pipelined datapath to handle the load instruction properly. The write register number now comes from the MEM/WB pipeline register along with the data. The register number is passed from the ID pipe stage until it reaches the MEM/WB pipeline register, adding five more bits to the last three pipeline registers. This new path is shown in color.

Diagram now shows the MEM/WB pipeline register data preserves the write register number.

Pipelined Control

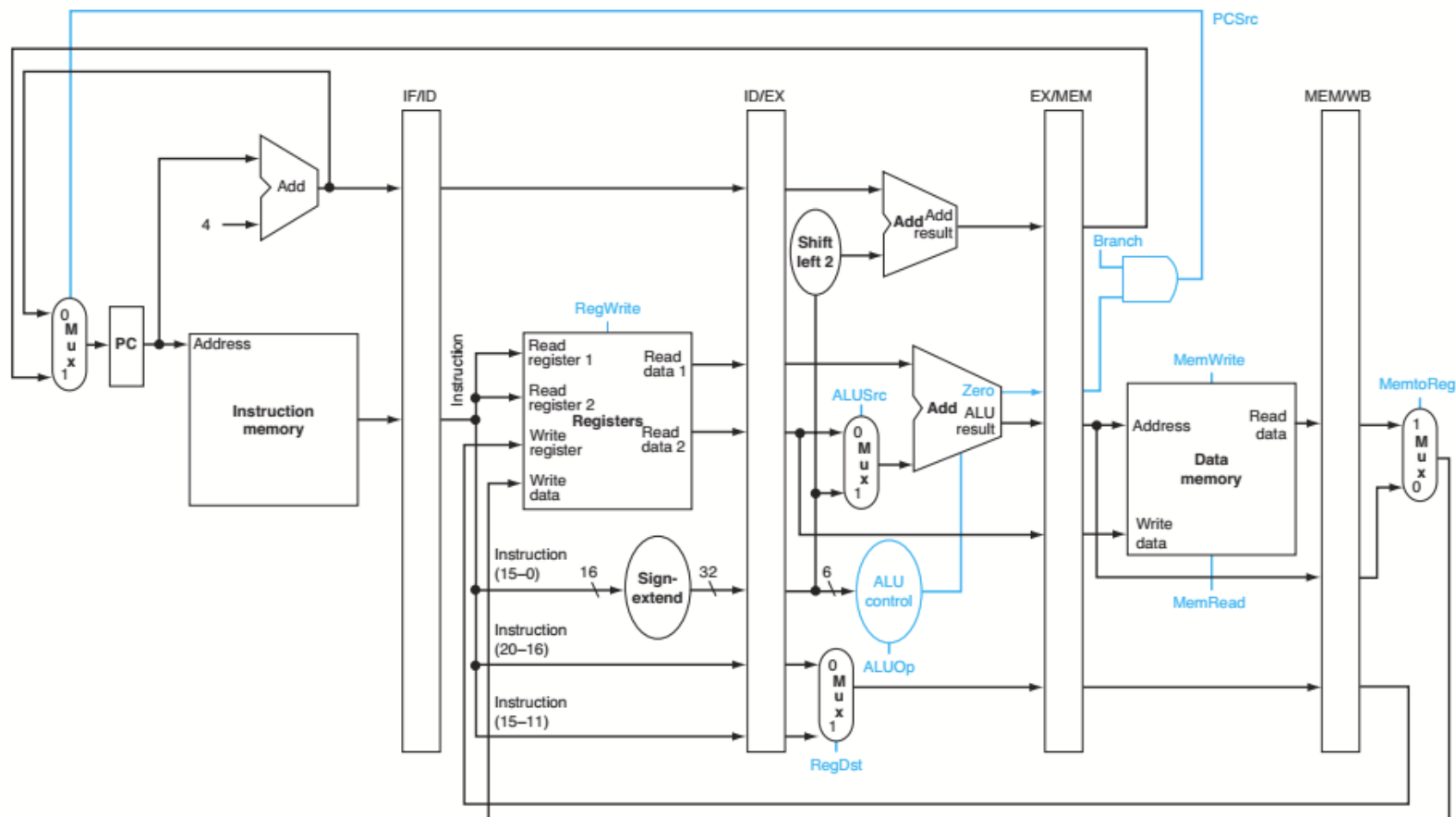


FIGURE 4.46 The pipelined datapath of Figure 4.41 with the control signals identified. This datapath borrows the control logic for PC source, register destination number, and ALU control from Section 4.4. Note that we now need the 6-bit funct field (function code) of the instruction in the EX stage as input to ALU control, so these bits must also be included in the ID/EX pipeline register. Recall that these 6 bits are also the 6 least significant bits of the immediate field in the instruction, so the ID/EX pipeline register can supply them from the immediate field since sign extension leaves these bits unchanged.

To specify control for the pipeline, we need only set the control values during each pipeline stage. Because each control line is associated with a component active in only a single pipeline stage, we can divide the control lines into five groups according to the pipeline stage.

1. *Instruction fetch*: The control signals to read instruction memory and to write the PC are always asserted, so there is nothing special to control in this pipeline stage.
2. *Instruction decode/register file read*: As in the previous stage, the same thing happens at every clock cycle, so there are no optional control lines to set.
3. *Execution/address calculation*: The signals to be set are RegDst, ALUOp, and ALUSrc (see Figures 4.47 and 4.48). The signals select the Result register, the ALU operation, and either Read data 2 or a sign-extended immediate for the ALU.

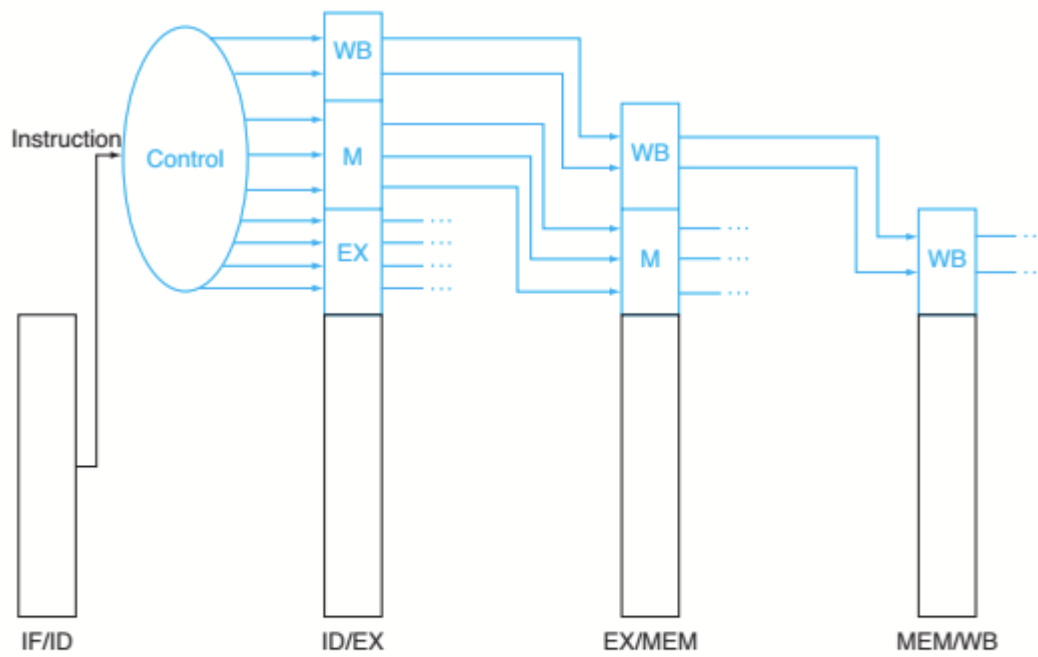


FIGURE 4.50 The control lines for the final three stages. Note that four of the nine control lines are used in the EX phase, with the remaining five control lines passed on to the EX/MEM pipeline register extended to hold the control lines; three are used during the MEM stage, and the last two are passed to MEM/WB for use in the WB stage.

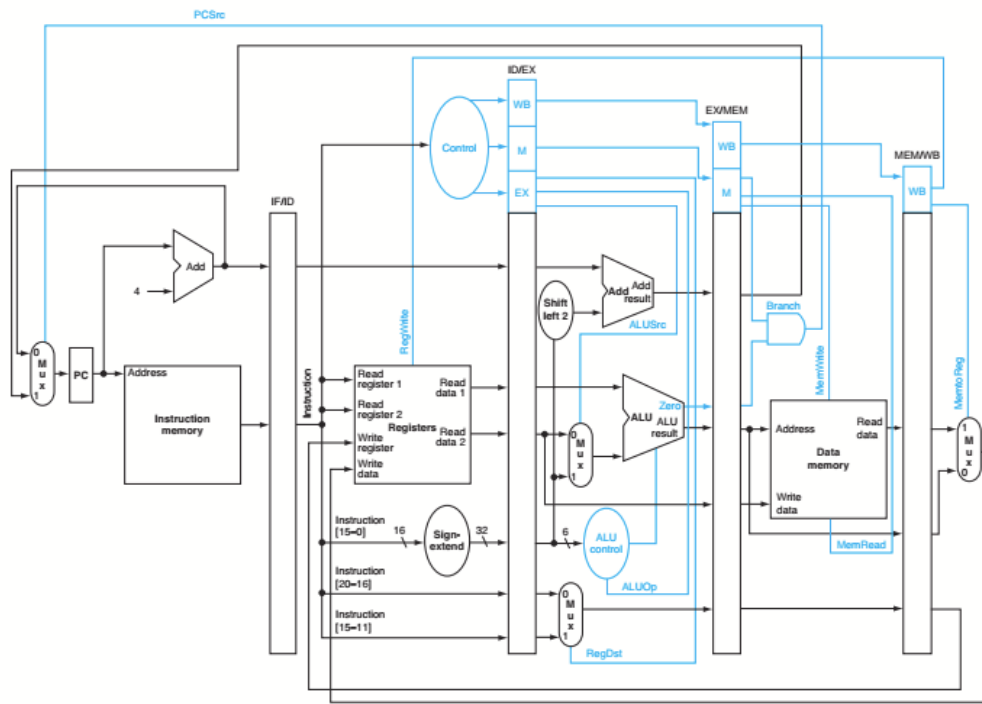


FIGURE 4.51 The pipelined datapath of Figure 4.46, with the control signals connected to the control portions of the pipeline registers. The control values for the last three stages are created during the instruction decode stage and then placed in the ID/EX pipeline register. The control lines for each pipe stage are used, and remaining control lines are then passed to the next pipeline stage.

There are 9 controls lines that have to be set each stage of operation.

7.5 Pipelined Processor

441

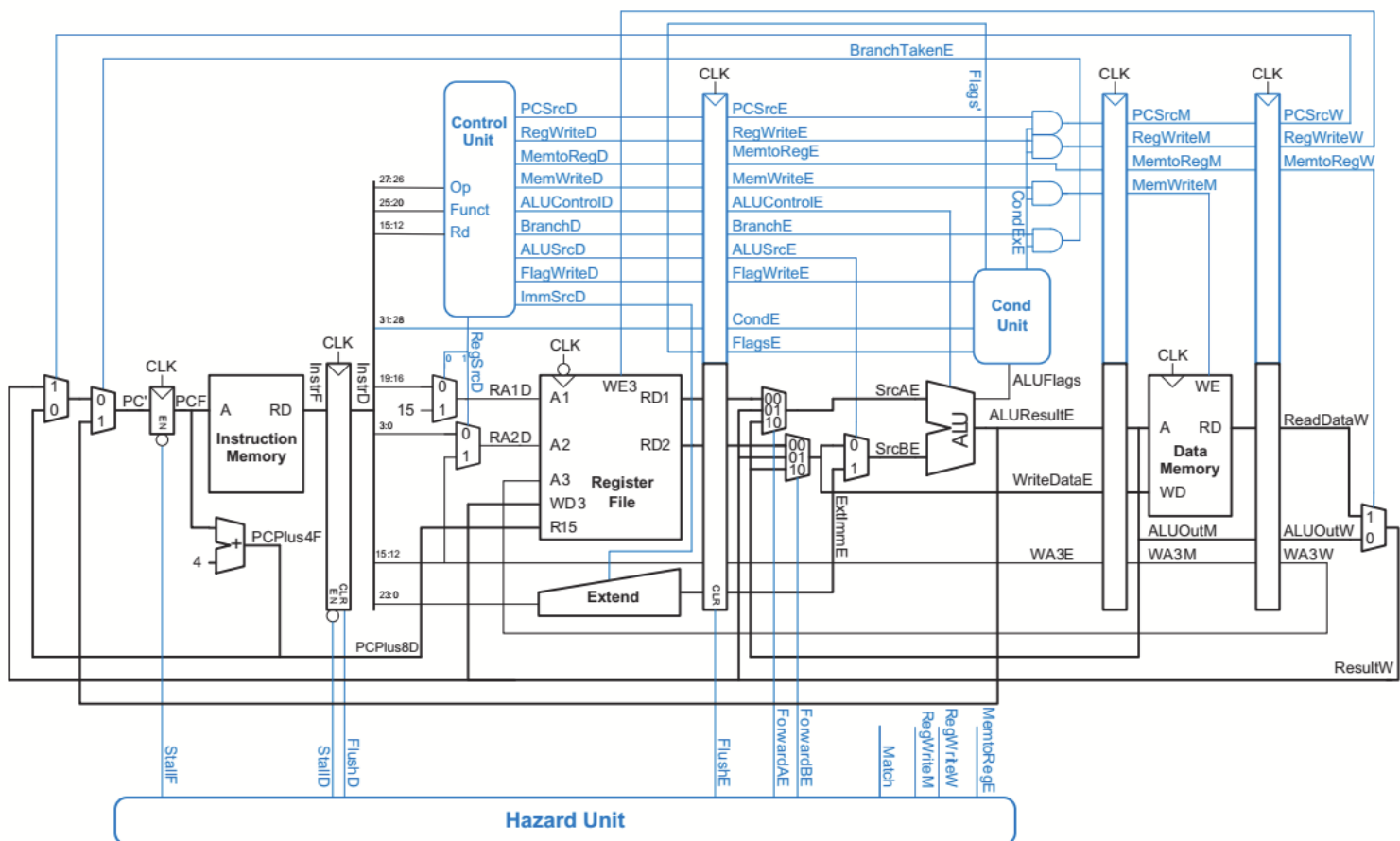


Figure 7.58 Pipelined processor with full hazard handling

Arm 5 stage pipeline with full hazard handling

Hazard Handling

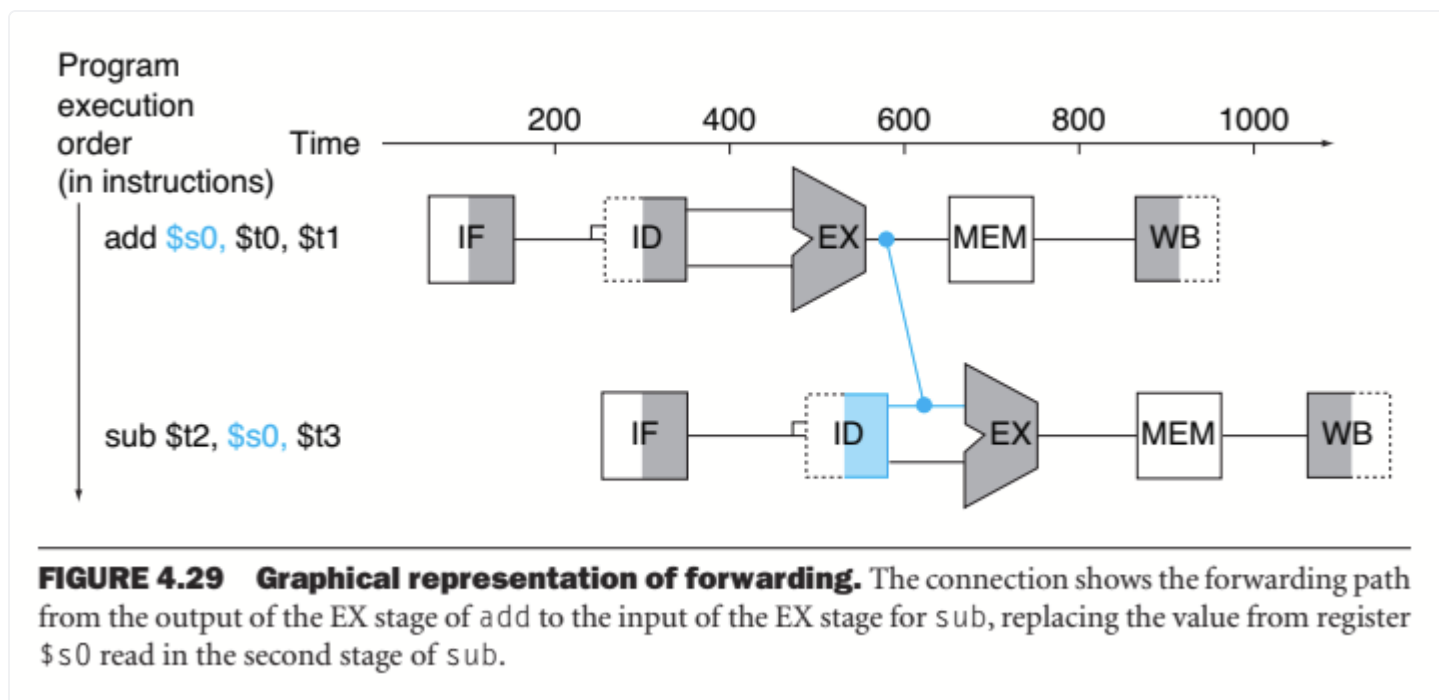
Structural Hazards

These types of hazards occur when the combination of instructions we want to perform cannot be executed in the same clock cycle since the hardware does not support it. (e.g. used a washer-dryer combo or roommate busy and can't put clothes away)

Data Hazards

Occur when the data is stalled due to a previous instructions

Say we have an add instruction followed by a subtract. The subtract would need to wait on the previous instruction for the sum. However, we can avoid this by using **forwarding** or **bypassing**.



forwarding paths only valid if the destination stage comes later than the source stage

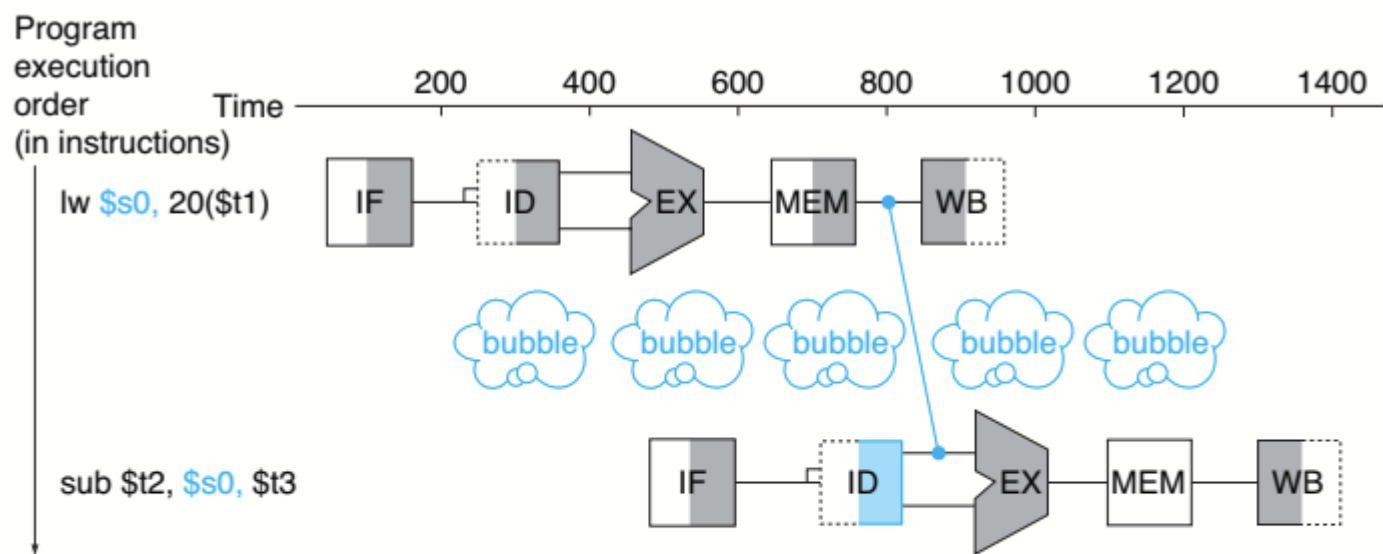


FIGURE 4.30 We need a stall even with forwarding when an R-format instruction following a load tries to use the data. Without the stall, the path from memory access stage output to execution stage input would be going backward in time, which is impossible. This figure is actually a simplification, since we cannot know until after the subtract instruction is fetched and decoded whether or not a stall will be necessary. [Section 4.7](#) shows the details of what really happens in the case of a hazard.

Limitations: forwarding does not prevent pipeline stalls when load is involved

Forwarding is harder to do if there are multiple results to forward per instruction or if they need to write a result earlier on in instruction execution.

What happens when register is read and written in same clock cycle?

We assume that write is in first half and read is in the second half, so read delivers what is written. This avoids the data hazard.

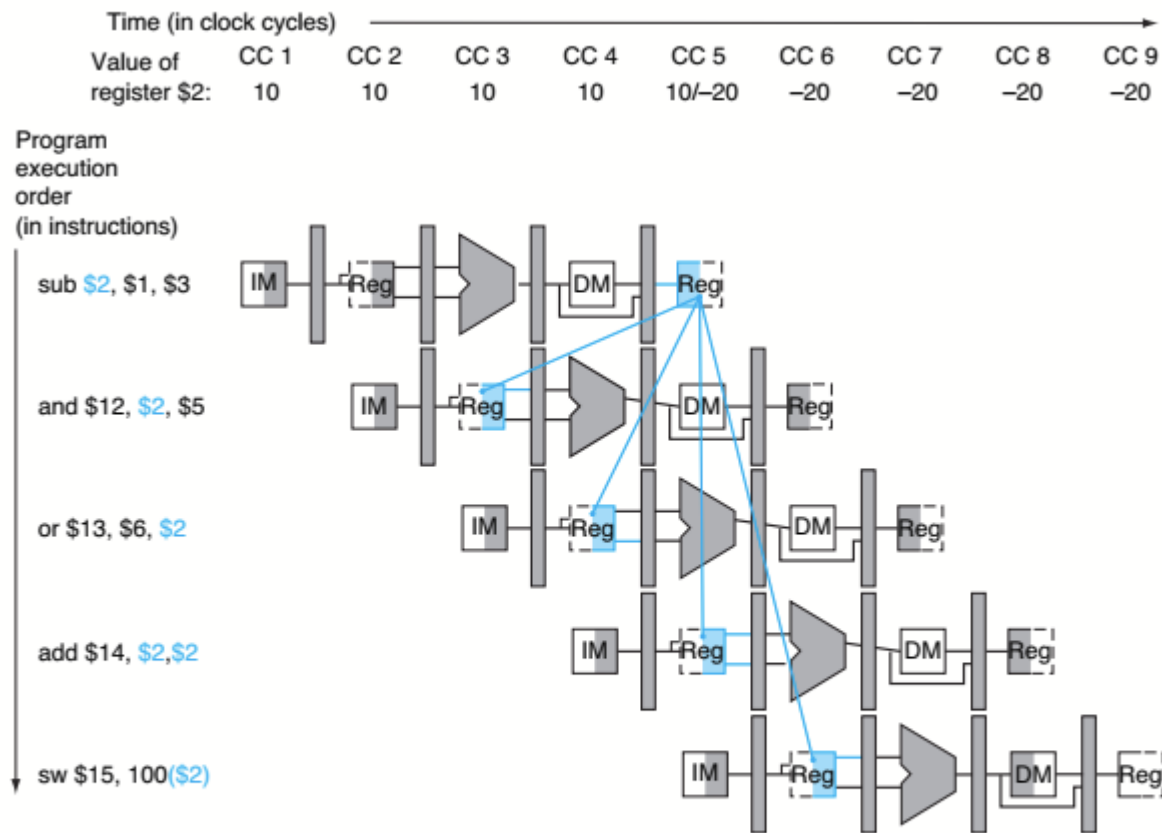


FIGURE 4.52 Pipelined dependences in a five-instruction sequence using simplified datapaths to show the dependences. All the dependent actions are shown in color, and “CC 1” at the top of the figure means clock cycle 1. The first instruction writes into \$2, and all the following instructions read \$2. This register is written in clock cycle 5, so the proper value is unavailable before clock cycle 5. (A read of a register during a clock cycle returns the value written at the end of the first half of the cycle, when such a write occurs.) The colored lines from the top datapath to the lower ones show the dependences. Those that must go backward in time are *pipeline data hazards*.

Another example of a data hazard. Only the add and sw would get the correct data as their EX stage occurs latter than the WB of sub.

Again to avoid the stalling we should simply forward data as soon as it is available at the beginning of the EX stage.

Pairs of hazard contitions:

- 1a. EX/MEM.RegisterRd = ID/EX.RegisterRs
- 1b. EX/MEM.RegisterRd = ID/EX.RegisterRt
- 2a. MEM/WB.RegisterRd = ID/EX.RegisterRs
- 2b. MEM/WB.RegisterRd = ID/EX.RegisterRt

Some instructions do not write registers so this policy doesn't always work. In this case we would have forwarding when it shouldn't. Solution is simply to check if the RegWrite signal will be active; this is examining the WB control field of the pipeline register during EX and MEM stages to see if the RegWrite is asserted.

If we taken inputs to the ALU from any pipeline register rather than ID/EX, we can forward the proper data. Should be able to add multiplexors to input of ALU to select between the register file value or the forward values.

Conditions for detecting hazards and control signals to resolve:

1. EX hazard:

```
if (EX/MEM.RegWrite
and (EX/MEM.RegisterRd != 0)
and (EX/MEM.RegisterRd = ID/EX. Register Rs))
ForwardA = 10
```

```
if (EX/MEM.RegWrite and (EX/MEM.RegisterRd != 0) and (EX/MEM.RegisterRd = ID/
EX.RegisterRt)) ForwardB = 10
```

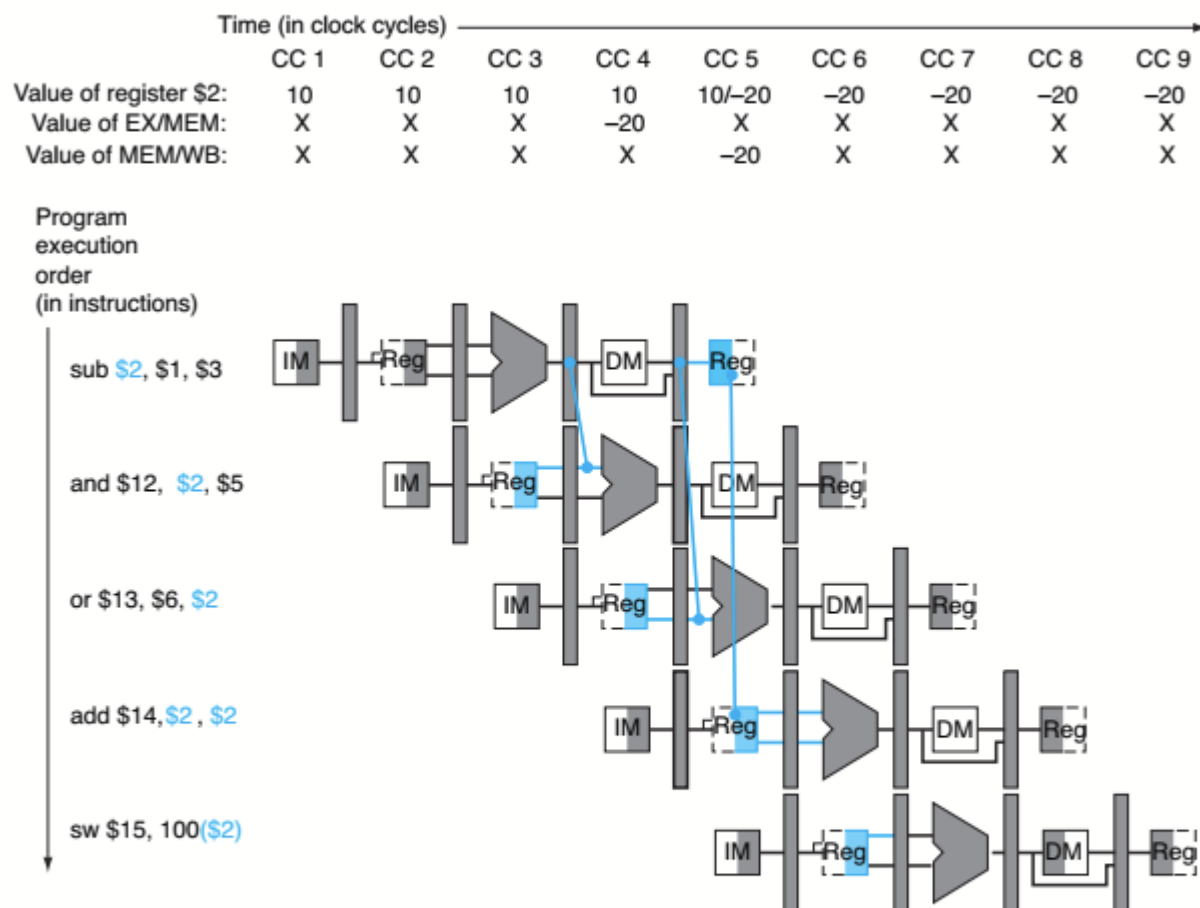
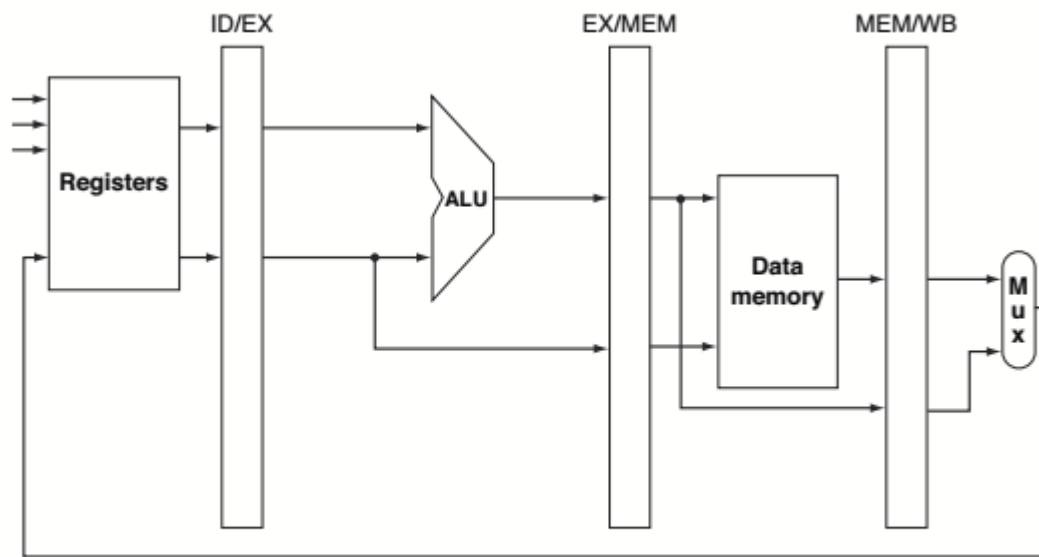
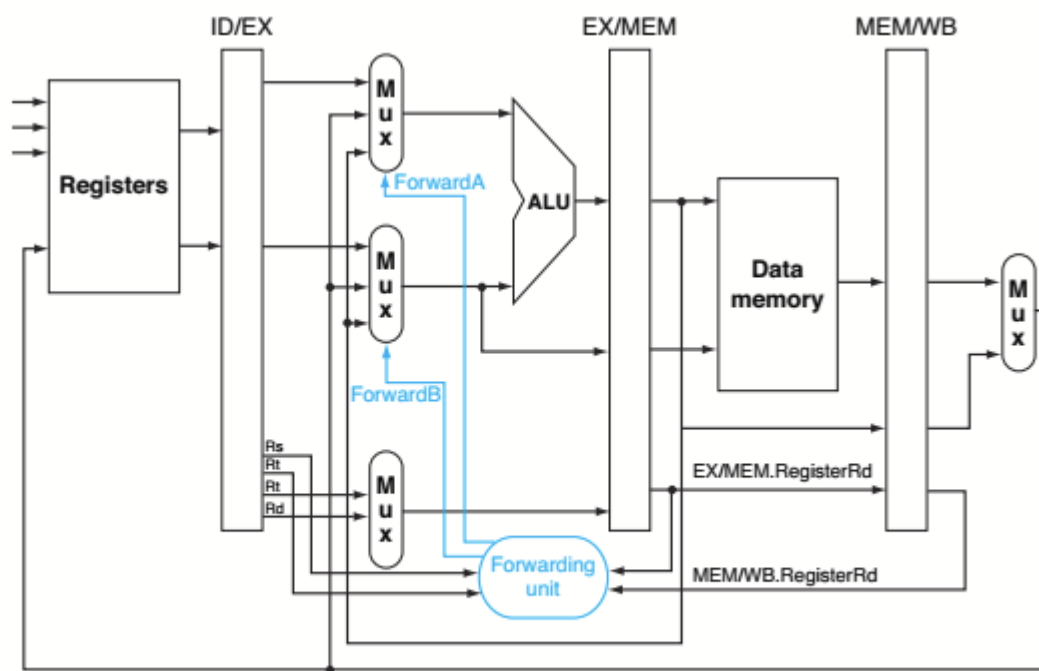


FIGURE 4.53 The dependences between the pipeline registers move forward in time, so it is possible to supply the inputs to the ALU needed by the AND instruction and OR instruction by forwarding the results found in the pipeline registers. The values in the pipeline registers show that the desired value is available before it is written into the register file. We assume that the register file forwards values that are read and written during the same clock cycle, so the add does not stall, but the values come from the register file instead of a pipeline register. Register file “forwarding”—that is, the read gets the value of the write in that clock cycle—is why clock cycle 5 shows register \$2 having the value 10 at the beginning and -20 at the end of the clock cycle. As in the rest of this section, we handle all forwarding except for the value to be stored by a store instruction.



a. No forwarding



b. With forwarding

2. MEM Hazard

if (MEM/WB.RegWrite and (MEM/WB.RegisterRd != 0) and (MEM/WB.RegisterRd = ID/EX.Register RS)) Forward = 01

if (MEM/WB.RegWrite and (MEM/WB.RegisterRd != 0) and (MEM/WB.RegisterRd = ID/EX.RegisterRt)) ForwardB = 01

Note: No hazards in the WB stage because we assume reg file supplies correct results if the instruction in ID stage reads same register written by instruction in WB stage.

(This is just to say that we read what was already written)

Potential hazards between result of instruction in WB stage, result of instruction in MEM stage, and source operand of instruction in ALU stage


```
add $1, $1, $2
add $1, $1, $3
add $1, $1, $4
```

Example: summing vector of numbers in a single register

Result would be forwarded from MEM stage because result in MEM stage is most recent.

```
if (MEM/WB.RegWrite and (MEM/WB.RegisterRd != 0) and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd != 0)) and
(EX/MEM.RegisterRd != ID/EX.RegisterRs)
and (MEM.WB.RegisterRd = ID/EX.RegisterRs)) ForwardA = 01

if (MEM/WB.RegWrite and (MEM/WB.RegisterRd != 0)
and not(EX/MEM.RegWrite and (EX/MEM.RegisterRd != 0)) and (EX/MEM.RegisterRd != ID/EX.RegisterRt)
and (MEM/WB.RegisterRd = ID/EX.ReisterRt)) ForwardB = 01
```

Mux control	Source	Explanation
ForwardA = 00	ID/EX	The first ALU operand comes from the register file.
ForwardA = 10	EX/MEM	The first ALU operand is forwarded from the prior ALU result.
ForwardA = 01	MEM/WB	The first ALU operand is forwarded from data memory or an earlier ALU result.
ForwardB = 00	ID/EX	The second ALU operand comes from the register file.
ForwardB = 10	EX/MEM	The second ALU operand is forwarded from the prior ALU result.
ForwardB = 01	MEM/WB	The second ALU operand is forwarded from data memory or an earlier ALU result.

FIGURE 4.55 The control values for the forwarding multiplexers in Figure 4.54. The signed immediate that is another input to the ALU is described in the *Elaboration* at the end of this section.

register destination for either an ALU instruction (which comes from the Rd field of the instruction) or a load (which comes from the Rt field).

Section 4.12 on the CD shows two pieces of MIPS code with hazards that cause forwarding, if you would like to see more illustrated examples using single-cycle pipeline drawings.

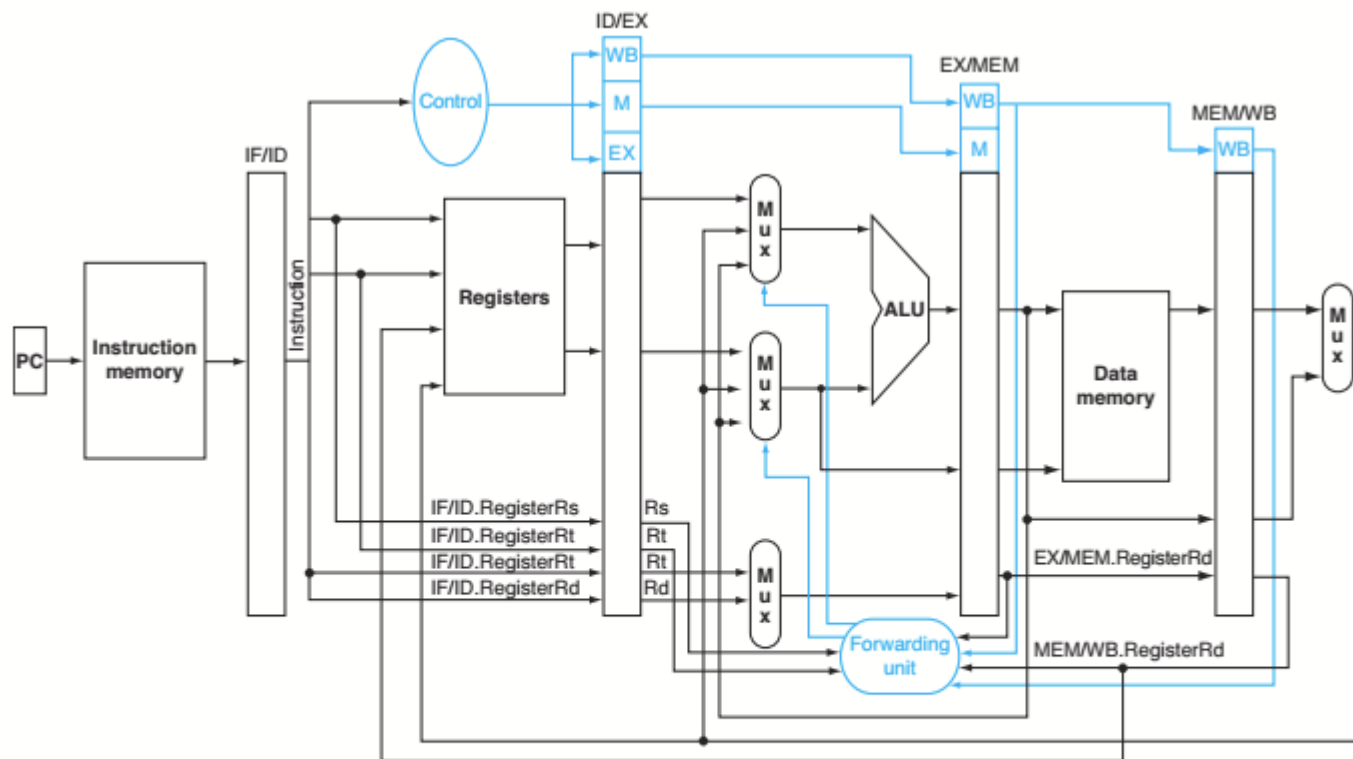


FIGURE 4.56 The datapath modified to resolve hazards via forwarding. Compared with the datapath in Figure 4.51, the additions are the multiplexers to the inputs to the ALU. This figure is a more stylized drawing, however, leaving out details from the full datapath, such as the branch hardware and the sign extension hardware.

Loads immediately followed by stores are useful when performing memory-to-memory copies in MIPS. Forwarding hardware would make them run quicker

Data Hazards and Stall

Forwarding will not work when we try to read a register following a load instruction that writes to the same register

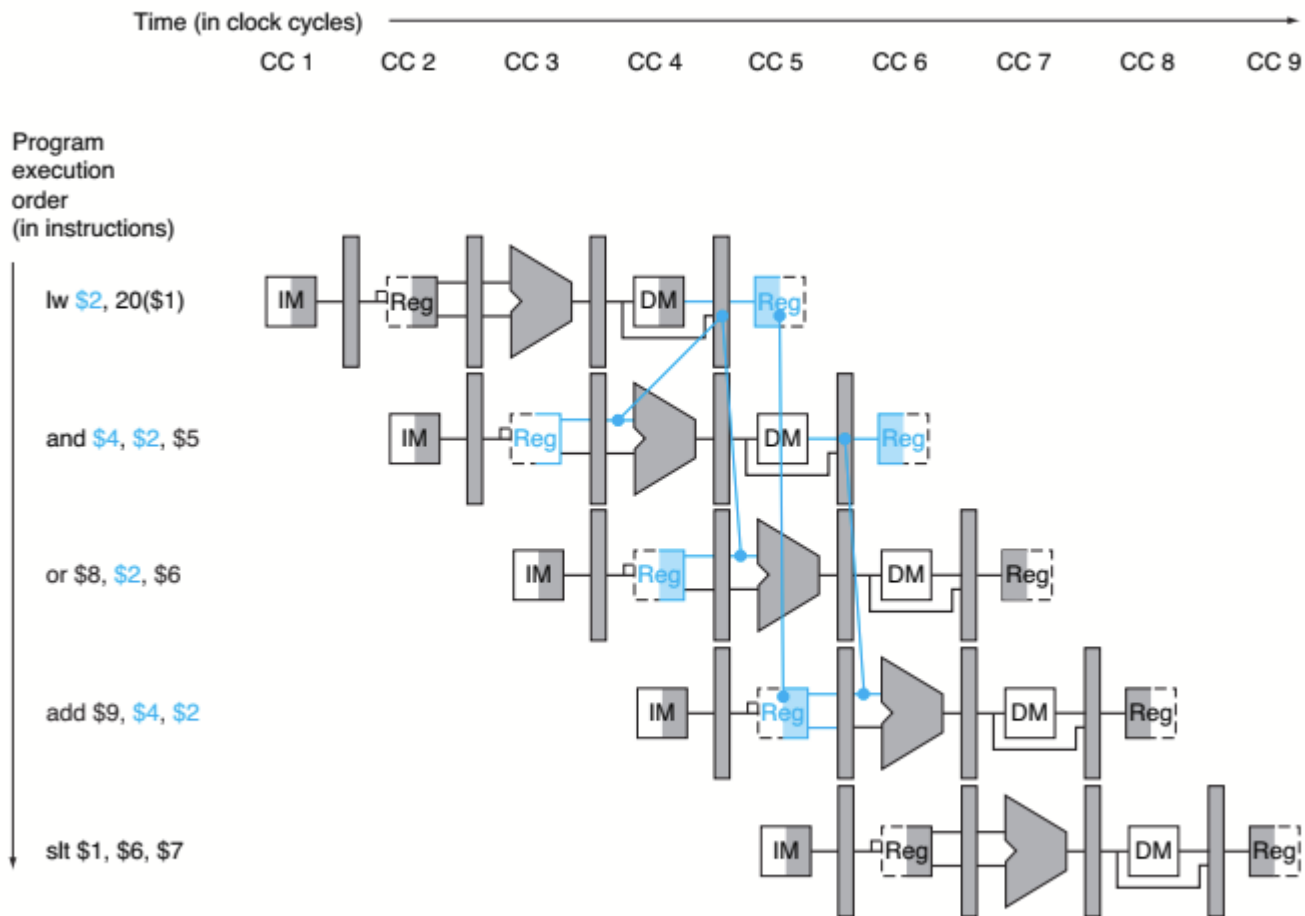


FIGURE 4.58 A pipelined sequence of instructions. Since the dependence between the load and the following instruction (and) goes backward in time, this hazard cannot be solved by forwarding. Hence, this combination must result in a stall by the hazard detection unit.

Data is still being read from memory in clock cycle 4 while ALU in following instruction is trying to perform the operation but doesn't have the up to date value yet. Therefore stall must be added to pipeline which needs a hazard detection unit.

```
if (ID/EX.MemRead and ((ID/EX.RegisterRt = IF/ID.RegisterRs) or (ID/EX.RegisterRt = IF/ID.RegisterRt)))
    stall the pipeline
```

Preventing these two instructions from making progress is accomplished simply by preventing the PC register and the IF/ID pipeline register from changing.

nops: an instruction that does no operation to change state

How to insert nops (bubbles) into the pipeline?

Deassert all nine control signals (set them to 0) creates a "do nothing" instruction.

By identifying the hazard in the ID stage, we can insert a nop bubble into the pipeline.

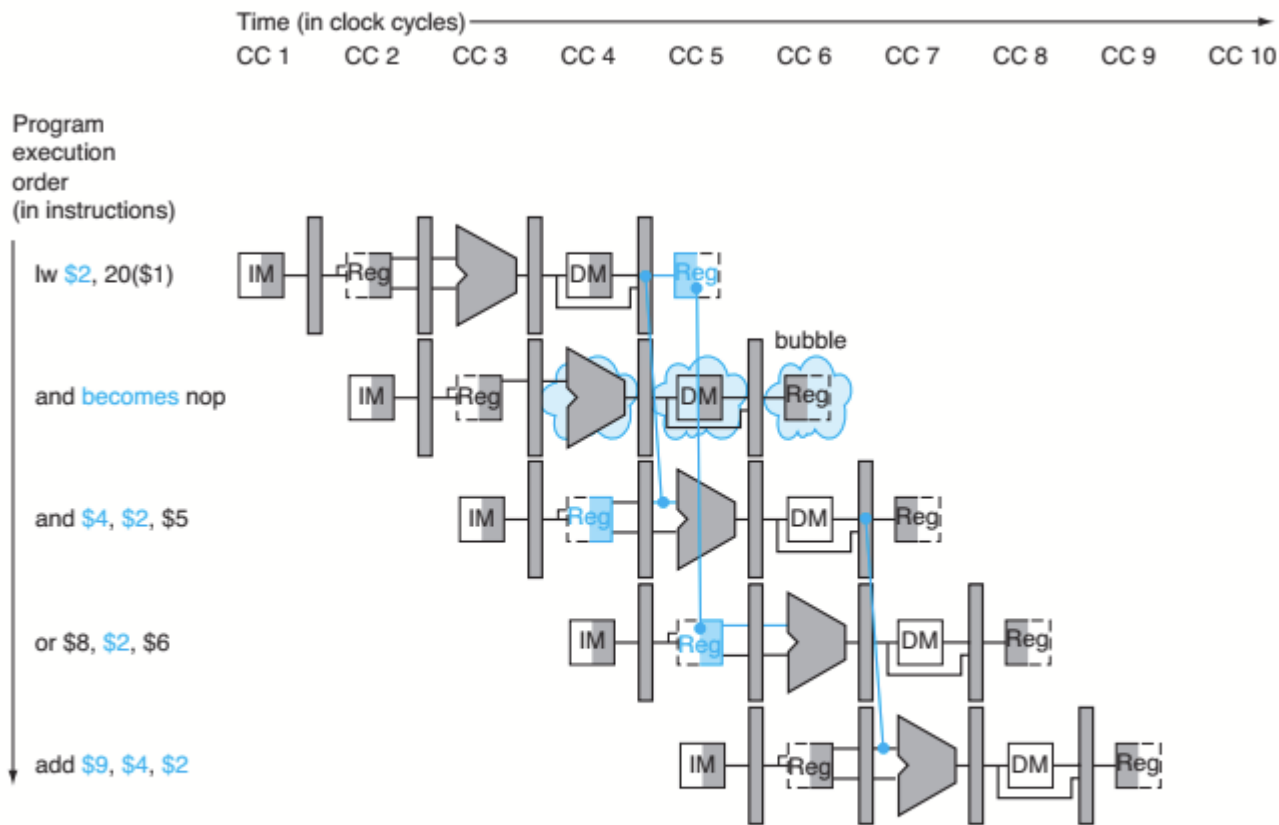


FIGURE 4.59 The way stalls are really inserted into the pipeline. A bubble is inserted beginning in clock cycle 4, by changing the `and` instruction to a `nop`. Note that the `and` instruction is really fetched and decoded in clock cycles 2 and 3, but its EX stage is delayed until clock cycle 5 (versus the unstalled position in clock cycle 4). Likewise the `OR` instruction is fetched in clock cycle 3, but its ID stage is delayed until clock cycle 5 (versus the unstalled clock cycle 4 position). After insertion of the bubble, all the dependences go forward in time and no further hazards occur.

Compiler must understand pipeline to achieve best performances. Otherwise unexpected stalls reduce performance of the compiled code.

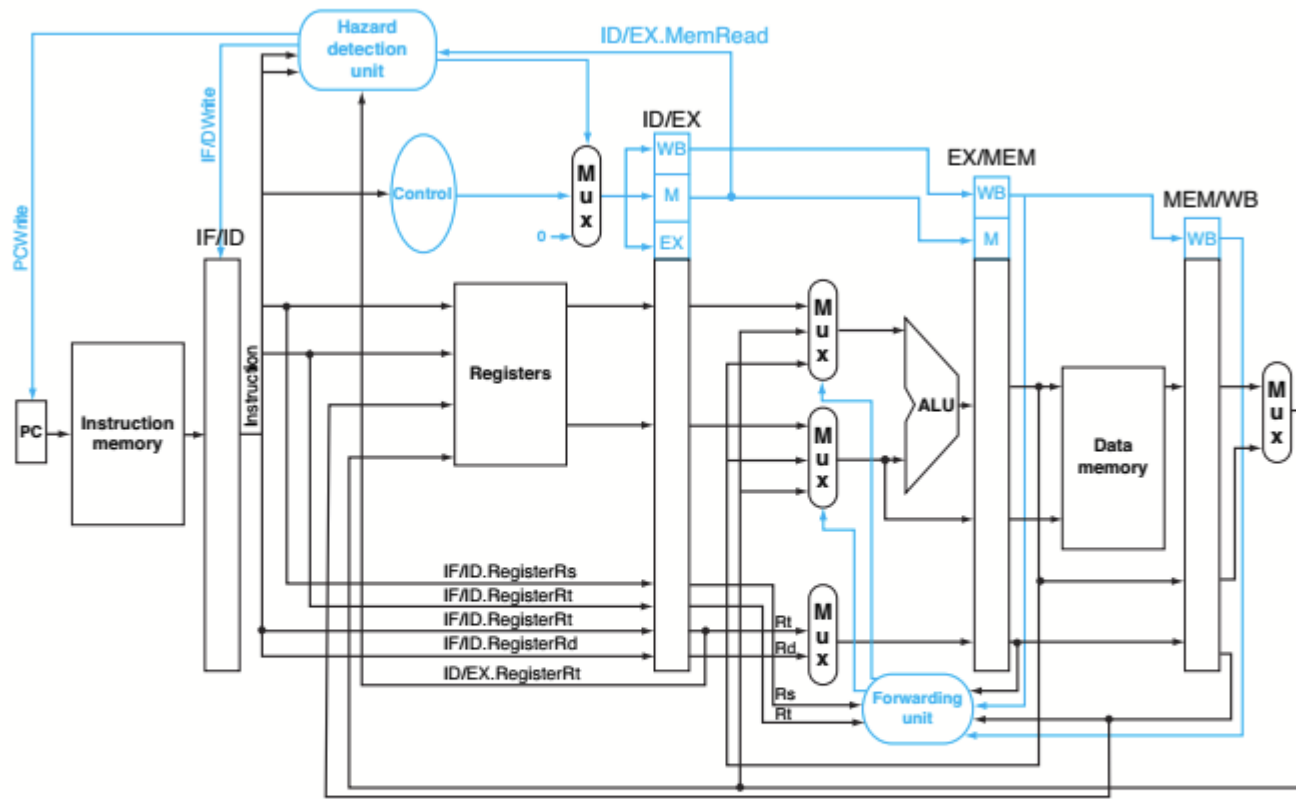


FIGURE 4.60 Pipelined control overview, showing the two multiplexers for forwarding, the hazard detection unit, and the forwarding unit. Although the ID and EX stages have been simplified—the sign-extended immediate and branch logic are missing—this drawing gives the essence of the forwarding hardware requirements.

Elaboration: Regarding the remark earlier about setting control lines to 0 to avoid writing registers or memory: only the signals RegWrite and MemWrite need be 0, while the other control signals can be don't cares.

Control Hazards

Happens when the flow of addresses is not what the pipeline expected.

control hazards occur in pipelined processors when the **outcome of a branch** is not known early enough to **determine the next instruction**.

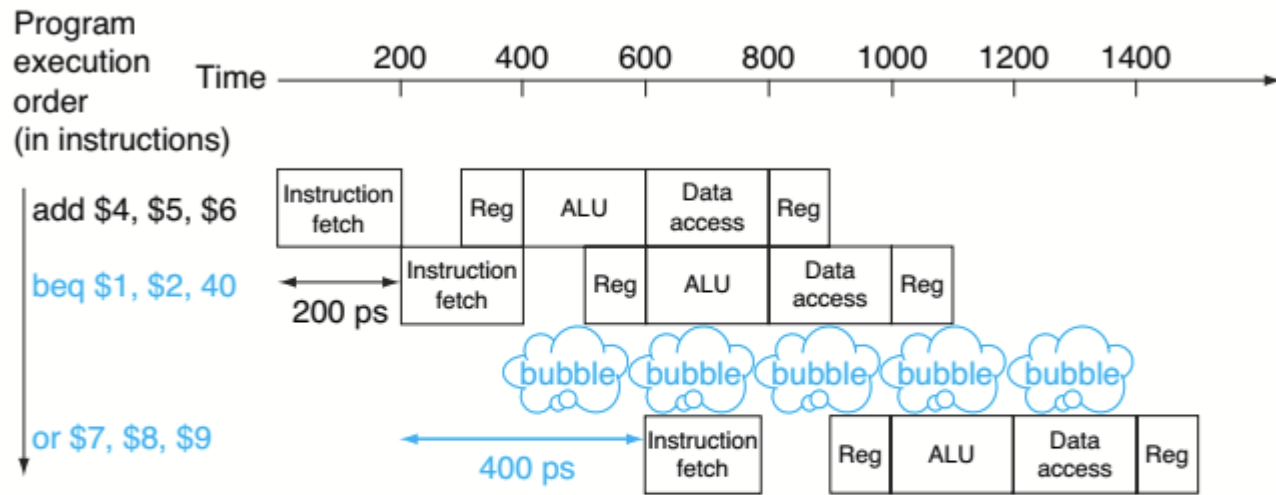


FIGURE 4.31 Pipeline showing stalling on every conditional branch as solution to control hazards. This example assumes the conditional branch is taken, and the instruction at the destination of the branch is the OR instruction. There is a one-stage pipeline stall, or bubble, after the branch. In reality, the process of creating a stall is slightly more complicated, as we will see in [Section 4.8](#). The effect on performance, however, is the same as would occur if a bubble were inserted.

One solution is prediction. If you have an idea of the expected outcome then you won't have to stall your guess. If your prediction fails though, you will have to redo it.

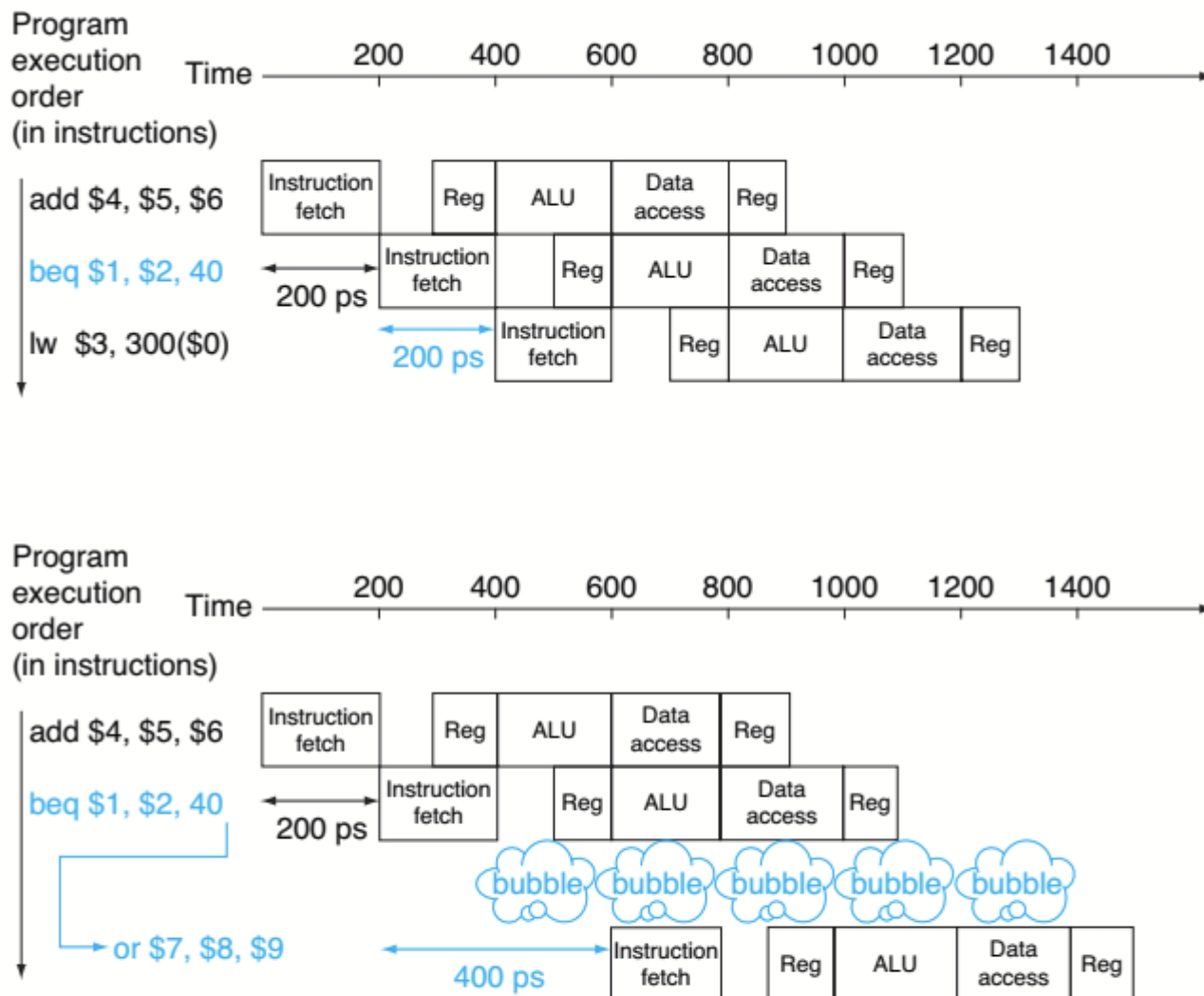


FIGURE 4.32 Predicting that branches are not taken as a solution to control hazard. The top drawing shows the pipeline when the branch is not taken. The bottom drawing shows the pipeline when the branch is taken. As we noted in Figure 4.31, the insertion of a bubble in this fashion simplifies what actually happens, at least during the first clock cycle immediately following the branch. Section 4.8 will reveal the details.

Dynamic Prediction

Approach: keeping a history of paths for each branch as taken and untaken.

Dynamic predictors shown to guess with more than 90% accuracy

When guess is wrong must make sure guess has no effect on and must restart pipeline from the proper branch address

Longer pipelines exacerbate problem since you have to start further back!

Delayed Branch

Approach: "while waiting for football clothes to dry, place non-football clothes in the washer until then"

Basically delay branch instructions by delaying by executing the next sequential instruction first.

This approach is used by MIPS

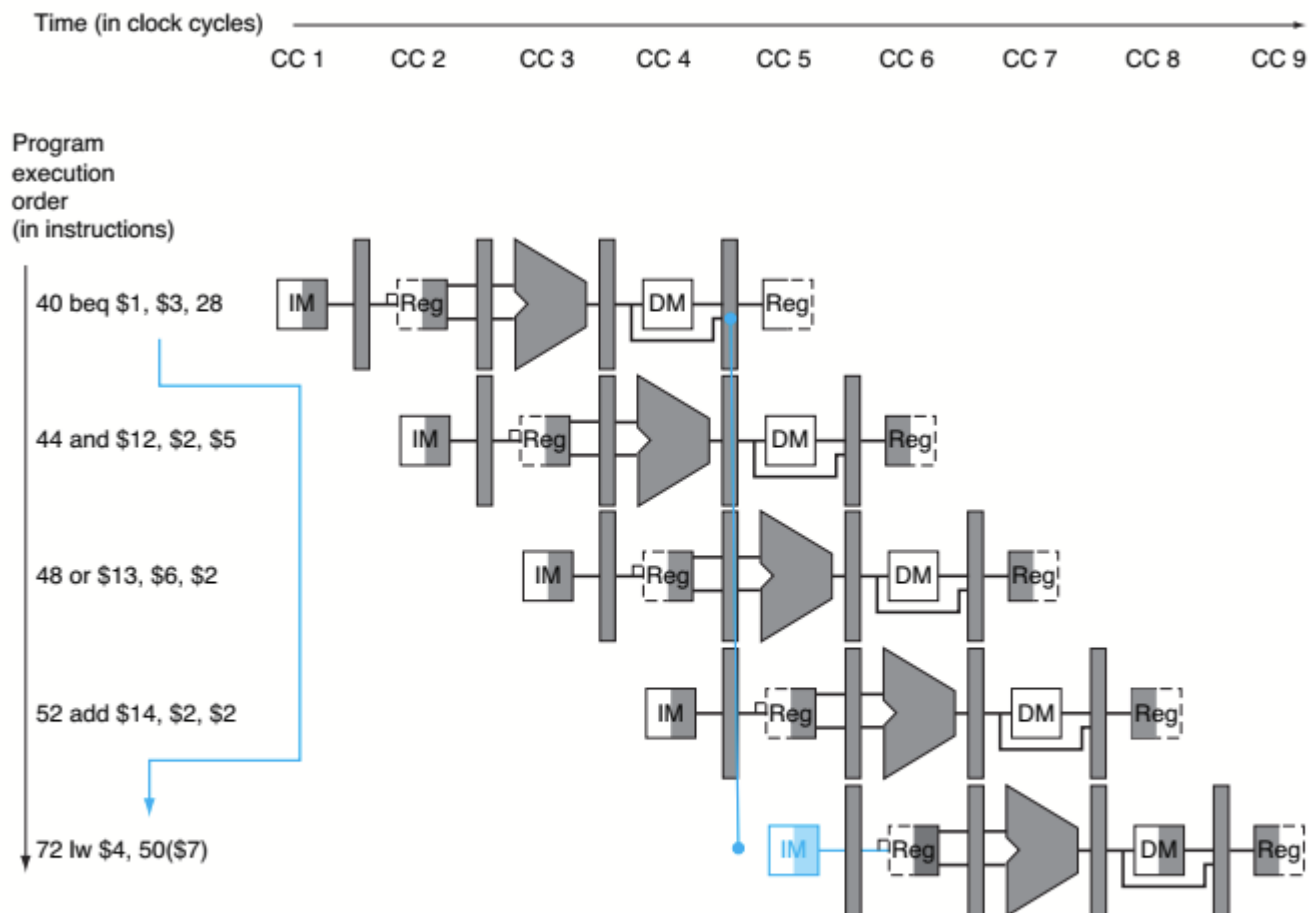


FIGURE 4.61 The impact of the pipeline on the branch instruction. The numbers to the left of the instruction (40, 44, ...) are the addresses of the instructions. Since the branch instruction decides whether to branch in the MEM stage—clock cycle 4 for the beq instruction above—the three sequential instructions that follow the branch will be fetched and begin execution. Without intervention, those three following instructions will begin execution before beq branches to lw at location 72. (Figure 4.31 assumed extra hardware to reduce the control hazard to one clock cycle; this figure uses the nonoptimized datapath.)

Delay in determining the proper instruction to fetch is called a control hazard

Relatively simple to understand, occur less frequently than data hazards. Nothing as effective against control hazards as forwarding is against data hazards .

Overview of simpler schemes

Assume branch Not Taken

Stalling until branch is complete is way too slow

This approach assumes branch will not be taken and continue execution down sequential instruction stream

If branch is taken must discard following executions and start over.

Idea if the branches are untaken half the time and if it costs little to discard instructions then this approach halves cost of control hazards

Reducing the Delay Of branches

reduce the cost of the taken branch

We assumed next PC for branch is selected in MEM stage. But if we moved branch execution earlier in pipeline then, less instructions need to be flushed.

MIPS supports fast single-cycle branches that could be pipelined with small branch penalty

How to move branch decision? Requires two actions to occur earlier.

1. computing branch target address (this is easy)

- a. We already have pC value and immediate field in the IF/ID pipeline registers. Just the branch addder from EX to the ID stage.
2. evaluating branch decision (harder)
 - a. branch equal: compare two registers read during ID stage.
 - b. Equality tested y XOR respective bits and ORing all of the results
 - c. Moving branch test to ID stage implies additional forwarding/hazard detection since branch depends on result still in pipeline.

Further difficulties with this approach

1. ID → decode instruction → decide whether a bypass to equality is needed → complete equality comparison so if instruction is branch → set PC to branch target address
2. Because values in branch comp are needed during ID but may be produced later in time, possible a data hazard can occur and stall will be neede d
 - a. IF load is immediately followed by a branch that is on load result, two stall cycles needed since result appears at end of MEM cycle but needed at beginning of ID for branch

To flushing instructions in IF, we add a control line called IF.Flush

- zeros instruction field of IF/ID pipeline register
- clearing registers transforms fetched instruction → **nop**

Dynamic Branch Prediction

- static prediction may not be adequate for deeper pipelines as the penalty for a wrong guess are higher
- Dynamic branch prediction relies on program execution

Branch prediction buffer

- small memory indexed by in lower portion of address of branch instruction.
- Contains bit that says whether the branch was recently taken or not.
- If prediction is wrong, incorrectly predicted instructions are deleted and prediction bit is inverted. Proper sequence is fetched and executed
- Shortcoming. Even if branch is almost always taken, we can predict incorrectly twice before recorrecting

2 bit prediction schemes are used often to match taken branch frequency for these highly regular branches

- A prediction must be wrong twice before it is changed.

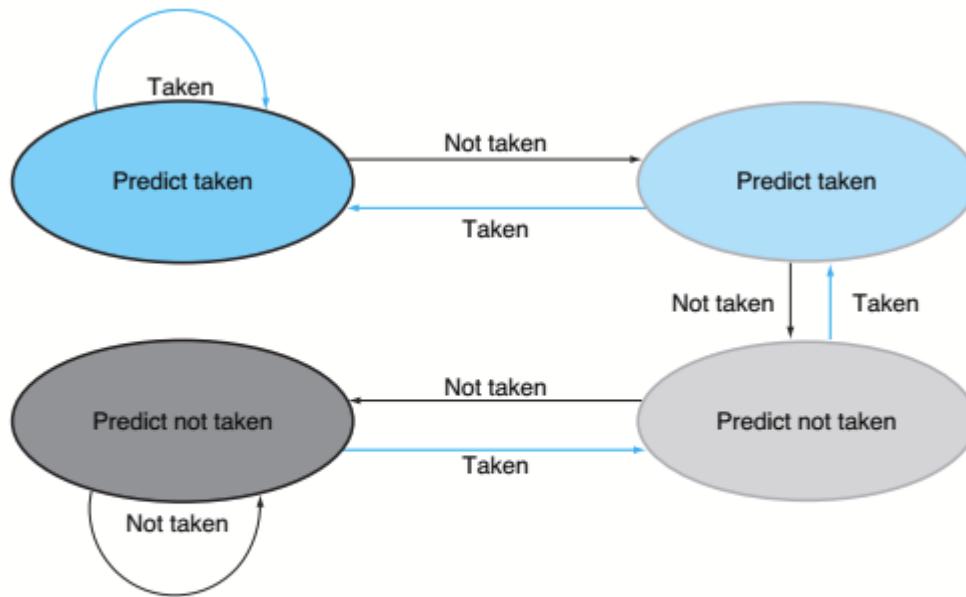


FIGURE 4.63 The states in a 2-bit prediction scheme. By using 2 bits rather than 1, a branch that strongly favors taken or not taken—as many branches do—will be mispredicted only once. The 2 bits are used to encode the four states in the system. The 2-bit scheme is a general instance of a counter-based predictor, which is incremented when the prediction is accurate and decremented otherwise, and uses the midpoint of its range as the division between taken and not taken.

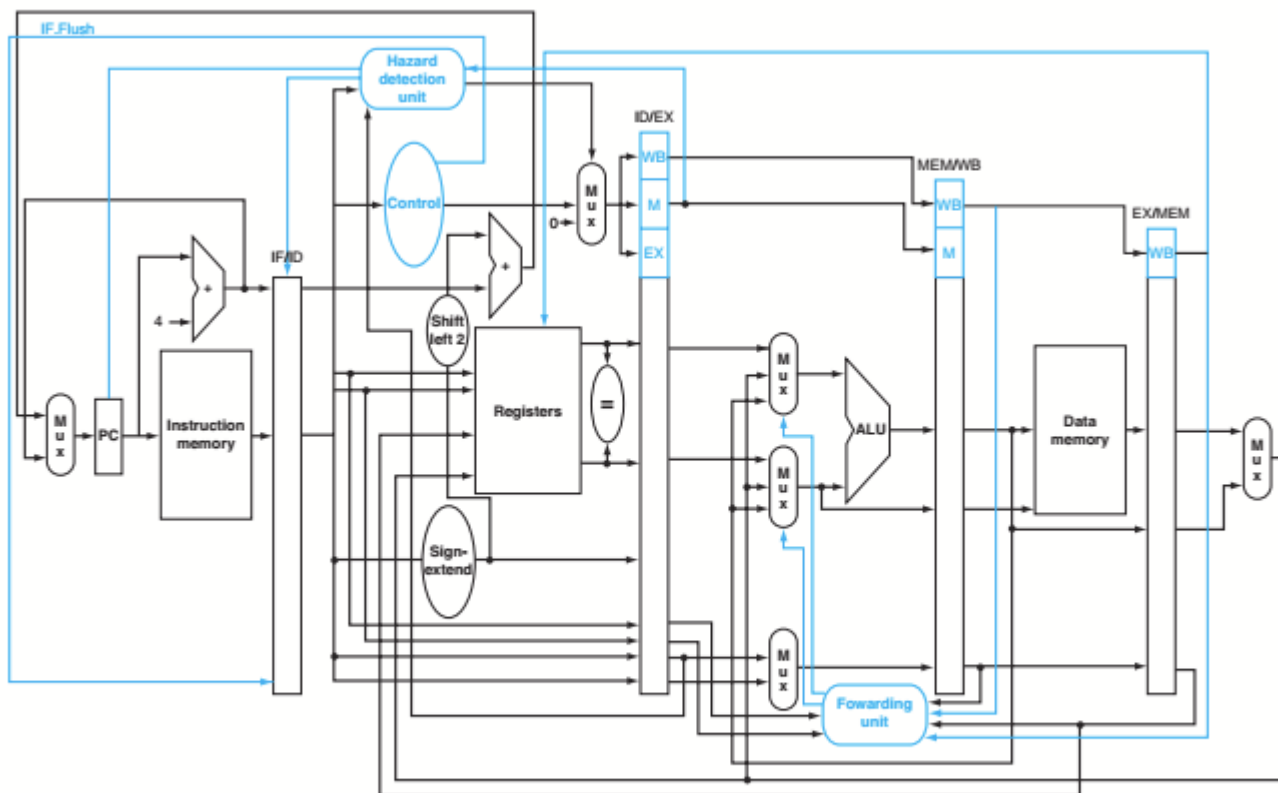
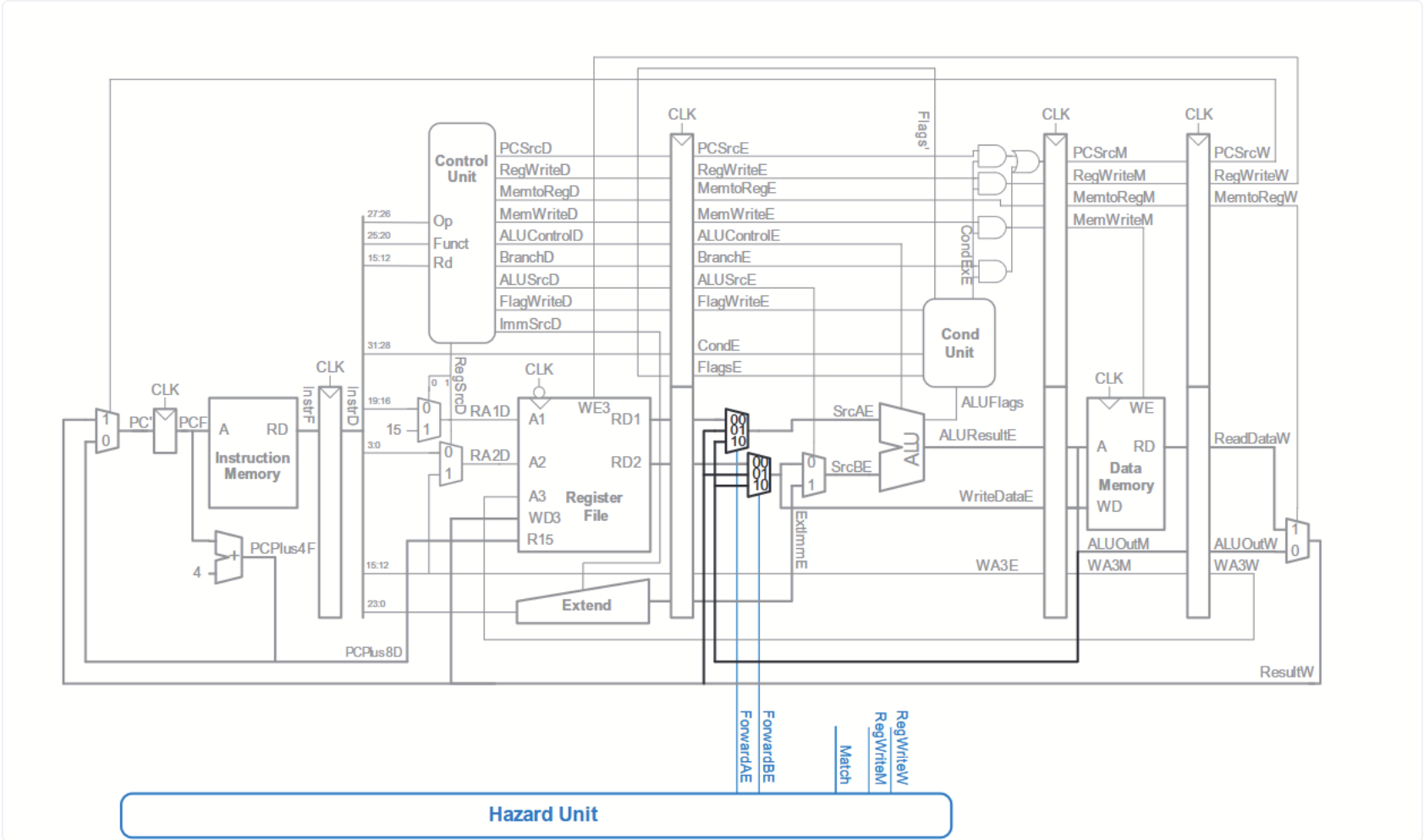


FIGURE 4.65 The final datapath and control for this chapter. Note that this is a stylized figure rather than a detailed datapath, so it's missing the ALUsrc mux from Figure 4.57 and the multiplexor controls from Figure 4.51.

ARM ISA



Little Endian

Instruction Set Architecture

Final Project.pdf

Specifications

- Target Market: General Purpose Computer
- Memory Organization: Von Neumann architecture
- Address Bus Width: 14 bits
- Number of Addresses 16384
- Program Load Address: 0x2000
- Data Bus Width: 36 bits
- Number of Register File Address Bits: 2
- Number Register File Registers: 4
- Width of Register File Register : 36
- Instruction Width: 18 bits
- Size of Immediate Field: 8 bits
- Maximum number of Instructions: 16
- Memory Addressing Endianness : Little

R-type	Op code (4 bits)	Destination Reg (2 bits)	Source Reg1 (2 bits)	Source Reg2 (2 bits)	Function (8 bits)
I-type	Op code (4 bits)	Destination Reg (2 bits)	Source Reg1 (2 bits)	Unused (2 bits)	Immediate (8 bits)
J-type	Op code (4 bits)	Immediate field (14 bits)			

Figure 1: Instruction formats and bit fields

This instruction format takes inspiration from the MIPS instruction set which classifies instructions into 3 classes: R, I, and J. R type instructions are defined as logic and arithmetic instructions (ADD, SUB, OR, AND). Meanwhile, I type instructions are known as "memory commands." In other words, they directly manipulate data memory (STORE/LOAD). Lastly, J-type instructions are referred to as branching instructions. For now, this instruction class will include BNE (Branch Not Equals) and an unconditional jump (JMP).

NEED to update because we use function bits

Table 2: Instruction Opcodes

Instruction	Opcode
Arithmetic (R type)	
add	0000
sub	0001
subs	0010
and	0011
or	0100
Load/Store (I type)	
load	0100
store	0101
addi	0110
subi	0111
Branching (J type)	
jmp	1000
bne	1001

Pipeline Registers

The datapath will include a 5-stage pipeline where instructions and data are passed through the following stages:

- IF/ID (instruction fetch/decode)
- ID/EX (instruction decode/ execution)
- EX/MEM (execution/memory)
- MEM/WB (memory/write back)

IF/ID pipeline register will be a 32 bit register as the instruction memory is 14 bits and instructions are 18 bits wide.

Swartz told me to look into building an ARM pipeline. Directed me to his videos and articles on his personal website.

Instead I will try to implement a 3-stage ARM pipeline with the following stages:

- IF/ID
- ID/EX
- EX/MEM/WB

ALU Design

The ALU determines the arithmetic and logic operations the computer should execute based on the controls signals obtained from the Instruction Decode Stage.

Earlier we defined the following set of instructions (to be expanded in the future)

Instruction	Opcode
Arithmetic (R type)	
add	0000
sub	0001
and	0010
or	0011
Load/Store (I type)	
load	0100
store	0101
addi	0110
subi	0111
Branching (J type)	
jmp	1000
bne	1001

Therefore our basic ALU should be able to perform add, sub, and, or operations. For now we can use a 2 bit control signal (ALUControls) as the input to the ALU's multiplexors.

ALUControls	Function
00	add
01	sub
10	and
11	or

ALU's are combinatorial circuits therefore we do not have to worry about sequential logic or flipflops. I think the ALU can be implemented as a series of case statements which should simplify a lot of the verilog.

